Gym Tracker Pro

Design Specification

Overview

GymTracker Pro follows the classical Spring MVC layered architecture deployed on Apache Tomcat. The architecture is organized into five distinct horizontal layers:

Layer	Technology	Responsibility
Presentation Layer	Thymeleaf, HTML, Bootstrap	Render HTML views; handle user input via forms; display validation errors.
Controller Layer	Spring MVC '@Controller'	Receive HTTP requests; delegate to service layer; select the view and populate the Model.
Service Layer	Spring '@Service' / '@Transactional'	Enforce business rules; orchestrate between repositories; manage transactions.
Repository Layer	Spring Data JPA '@Repository'	Abstract database access; provide CRUD and custom queries via JPQL.
Persistence Layer	Hibernate ORM, MySQL	Map Java entities to relational tables; manage schema via DDL.

Configuration is entirely XML/annotation-based without Spring Boot auto-configuration:

- 'web.xml' - declares 'DispatcherServlet', 'ContextLoaderListener', character encoding filter, and Spring Security filter chain.
- 'applicationContext.xml' - defines DataSource bean, JPA EntityManagerFactory, TransactionManager, and component scan.
- dispatcher-servlet.xml - configures ViewResolver (Thymeleaf TemplateEngine), resource handlers, and MVC interceptors.

Package Structure

Package	Requirement
com.gymtracker.model	JPA entity classes: 'Member', 'WorkoutPlan', 'WorkoutSession', 'MemberPlan'
com.gymtracker.repository	JPA repository interfaces extending 'JpaRepository'
com.gymtracker.service	Business logic interfaces and implementations
com.gymtracker.controller	Spring MVC '@Controller' classes

com.gymtracker.dto	Data Transfer Objects for form binding and report data
com.gymtracker.validator	Custom Spring 'Validator' implementations
com.gymtracker.exception	Custom exception classes and global '@ControllerAdvice'
com.gymtracker.config	Spring configuration classes with '@Configuration'
com.gymtracker.util	Helper utilities: date formatters, report aggregators

Architecture and Requirements Traceability

Every functional requirement is traceable to at least one controller, service method, and repository:

ID	Controller	Service Method	Repository	View
FR-01	MemberController	MemberService.create()	MemberRepository.save()	members/create
FR-07	PlanController	PlanService.create()	PlanRepository.save()	plans/create
FR-12	SessionController	SessionService.log()	SessionRepository.save()	sessions/log
FR-18	MemberController	MemberService.assignPlan()	MemberPlanRepository.save()	members/assign-plan
FR-21	ReportController	ReportService.memberProgress()	SessionRepository.findByMemberAndDateBetween()	reports/member-progress
FR-22	ReportController	ReportService.facilityUsage()	SessionRepository.countByMonth()	reports/facility-usage

Error Handling Strategy

A coherent, application-wide error handling strategy is defined at three levels.

Validation Errors (User Input)

A coherent, application-wide error handling strategy is defined at three levels.

- Bean Validation (JSR-380) annotations are used on all DTO classes ('@NotBlank', '@Email', '@Min', '@Max', '@PastOrPresent').
- Controllers use '@Valid' and 'BindingResult' to capture validation errors and re-display the form with error messages alongside each field.
- Custom 'Validator' classes handle cross-field rules (e.g., session end date must be after start date).

Business Rule Violations

- Service layer throws typed custom exceptions: ‘ResourceNotFoundException’, ‘DuplicateEmailException’, ‘EntityInUseException’.
- A global ‘@ControllerAdvice’ class (‘GlobalExceptionHandler’) catches each exception type and maps it to a specific HTTP status and error view.
- Business exceptions are never allowed to propagate to the presentation layer as raw stack traces.

System / Infrastructure Errors

- ‘DataAccessException’ from Spring Data JPA is caught in ‘GlobalExceptionHandler’ and mapped to a generic ‘/error’ page.
- Custom 404 and 500 error pages are registered in ‘web.xml’ via ‘<error-page>’ declarations.
- All exceptions at ‘WARN’ level or above are written to the application log (SLF4J + Logback) with full stack trace, request URI, and timestamp.

Exception Class	HTTP Status	Handler	User Message
ResourceNotFoundException	404 Not Found	GlobalExceptionHandler	"The requested resource was not found."
DuplicateEmailException	409 Conflict	GlobalExceptionHandler	"This email address is already registered."
EntityInUseException	409 Conflict	GlobalExceptionHandler	"This record cannot be deleted as it is referenced by existing data."
MethodArgumentNotValidException	400 Bad Request	Spring MVC ‘@Valid’	Inline form-field error messages.
DataAccessException	500 Internal Error	GlobalExceptionHandler	"A system error occurred. Please try again later."

Design Patterns Applied

Pattern	Category	Application
MVC	Architectural	Core architectural pattern. Controllers handle HTTP, Services handle logic, Thymeleaf templates handle rendering. Enforced by Spring MVC.
Repository	Structural	All database access is abstracted behind repository interfaces

		(Spring Data JPA). Controllers and services never use 'EntityManager' directly.
Service Layer (Facade)	Structural	'MemberService', 'PlanService', 'SessionService', 'ReportService' each present a simplified API over multiple repositories, hiding transactional coordination.
DTO (Data Transfer Object)	Structural	Separate DTO classes ('MemberForm', 'SessionForm', 'ReportFilterDto') decouple the web form model from JPA entities, preventing over-posting attacks.
Template Method	Behavioral	'AbstractReportService' defines the report generation skeleton (filter, aggregate, format), with 'MemberProgressReportService' and 'FacilityReportService' providing concrete implementations.
Strategy	Behavioral	Report export format (HTML vs. PDF) is encapsulated as a 'ReportRenderer' strategy interface with 'HtmlReportRenderer' and 'PdfReportRenderer' implementations.
Dependency Injection	IoC	All dependencies are injected via constructor injection for testability and immutability.
Observer (Spring Events)	Behavioral	'ApplicationEventPublisher' publishes 'MemberCreatedEvent' and 'SessionLoggedEvent'. A listener updates aggregate statistics asynchronously.

Class and Interface Descriptions

Naming convention: [Domain][Layer] - e.g., MemberController, MemberService, MemberRepository.

Model (Entity) Classes

Class	Role and Key Fields
Member	'@Entity' Represents a gym member. Key fields: 'id' (PK), 'fullName', 'email' (unique), 'phone', 'dateOfBirth', 'membershipType' (enum), 'membershipStartDate', 'active' (boolean). Relations: 'OneToMany WorkoutSessions', 'OneToMany MemberPlans'.
WorkoutPlan	'@Entity' Represents a structured exercise programme. Key fields: 'id' (PK), 'name', 'description', 'difficultyLevel' (enum: 'BEGINNER'/'INTERMEDIATE'/'ADVANCED'), 'durationWeeks', 'targetMuscleGroups', 'createdAt', 'updatedAt'.

	Relations: ‘OneToMany WorkoutSessions’, ‘OneToMany MemberPlans’.
WorkoutSession	‘@Entity’ Records a single training session. Key fields: ‘id’ (PK), ‘sessionDate’, ‘durationMinutes’, ‘performanceRating’ (1–5), ‘notes’. Relations: ‘ManyToOne Member’, ‘ManyToOne WorkoutPlan’.
MemberPlan	‘@Entity’ (join entity). Links a ‘Member’ to a ‘WorkoutPlan’ assignment. Key fields: ‘id’, ‘member’ (FK), ‘workoutPlan’ (FK), ‘assignedDate’, ‘targetEndDate’, ‘active’ (boolean).

Repository Interfaces

Interface	Role and Key Methods
MemberRepository	Extends ‘JpaRepository<Member, Long>’. Custom: ‘findByEmail()’, ‘findByActiveTrue()’, ‘findByMembershipType()’, ‘countByActive()’.
WorkoutPlanRepository	Extends ‘JpaRepository<WorkoutPlan, Long>’. Custom: ‘findByDifficultyLevel()’, ‘existsByIdAndSessionsIsNotEmpty()’.
WorkoutSessionRepository	Extends ‘JpaRepository<WorkoutSession, Long>’. Custom: ‘findByMemberAndSessionDateBetween()’, ‘countByWorkoutPlan()’, ‘sumDurationByMember()’, ‘avgRatingByMemberAndDateBetween()’.
MemberPlanRepository	Extends ‘JpaRepository<MemberPlan, Long>’. Custom: ‘findActivePlanByMember()’, ‘findByMemberAndWorkoutPlan()’.

Service Classes

Class	Role and Business Logic
MemberServiceImpl	Handles all member lifecycle operations. Validates email uniqueness before create/update. Enforces soft-delete rule (blocks hard delete if sessions exist). Fires ‘MemberCreatedEvent’.
WorkoutPlanService	Manages workout plan CRUD. Validates referential integrity before

Impl	deletion. Throws ‘EntityInUseException’ if plan has sessions.
WorkoutSessionServiceImpl	Validates session date (not future) and duration bounds. Links session to member's active plan if none explicitly provided. Updates member aggregate stats.
MemberPlanServiceImpl	Manages plan assignment. Handles the "replace or retain" logic for concurrent active plans. Records assignment history.
ReportServiceImpl	Orchestrates report data assembly. Queries ‘SessionRepository’ with date range filters. Builds ‘MemberProgressReportDto’ and ‘FacilityReportDto’ for controller consumption.

Controller Classes

Class	Role and URL Mappings
MemberController	Handles all ‘/members/**’ routes. Provides: ‘GET /members’ (list), ‘GET /members/new’ (form), ‘POST /members’ (create), ‘GET /members/{id}’ (detail), ‘GET /members/{id}/edit’ (form), ‘PUT /members/{id}’ (update), ‘DELETE /members/{id}’ (soft delete).
PlanController	Handles all ‘/plans/**’ routes. Mirrors ‘MemberController’ structure for ‘WorkoutPlan’ CRUD.
SessionController	Handles ‘/sessions/**’ routes. Supports creation pre-populated with member and active plan. Also handles ‘/members/{id}/sessions’ for member-scoped session history.
ReportController	Handles ‘/reports/**’. ‘GET /reports/member/{id}’ accepts date range query params and renders ‘MemberProgressReport’. ‘GET /reports/facility’ renders ‘FacilityUsageReport’.
GlobalExceptionHandler	‘@ControllerAdvice’. Catches typed exceptions and maps to error views with appropriate HTTP status codes.

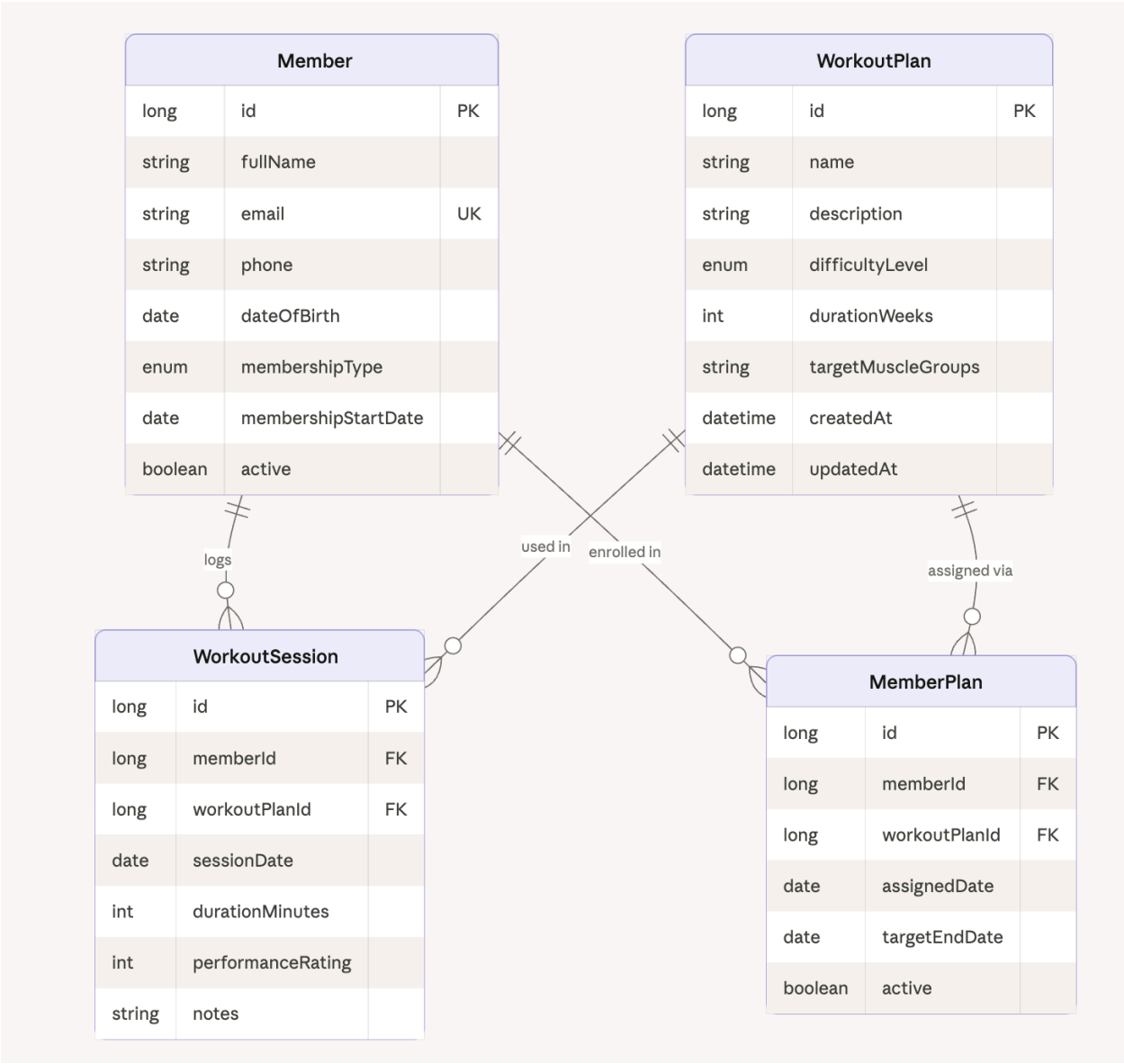
Design for Change & Extensibility

The architecture is explicitly designed to accommodate the following likely changes without requiring structural refactoring:

- New report types: The ‘AbstractReportService’ Template Method pattern means a new report type requires only a new concrete subclass implementing ‘aggregate()’ and ‘format()’, not changes to existing classes.
- REST API addition: Controllers are strictly separated from service logic. A new REST ‘@RestController’ layer can be added alongside the MVC controllers, consuming the same services, to expose a JSON API with zero service modification.
- Authentication upgrade: Spring Security is integrated via ‘FilterChain’ in ‘web.xml’. Switching from form login to OAuth2/LDAP requires only Security configuration changes, not application code changes.

- Database migration: The Repository pattern abstracts the database. Switching from MySQL to PostgreSQL requires only a JDBC driver swap and dialect property change. No service or controller code changes.
- UI framework swap: Controllers only interact with the Model map; they are View-agnostic. Replacing Thymeleaf with a React SPA requires only a 'ViewResolver' swap and converting controllers to '@RestController'.
- Additional entity fields: Hibernate schema auto-update handles additive column changes. DTO classes are separate from entities, so form changes do not cascade to the database model.
- New membership types: 'MembershipType' is an enum; adding a new value requires only the enum change and any relevant view update. No service logic change needed.

Data Model - Entity Relationships



Relationships

- A member can log many sessions; each session belongs to exactly one member.
- A plan can be used in many sessions; each session references exactly one plan.
- A member can be assigned multiple plans over time; a plan can be assigned to many members. 'MemberPlan' is the associative entity recording each assignment with dates.

Database Integrity Rules

Constraint	Role
WorkoutSession.member_id	NOT NULL, FK - Member.id, ON DELETE RESTRICT
WorkoutSession.plan_id	NOT NULL, FK - WorkoutPlan.id, ON DELETE RESTRICT
MemberPlan.member_id	NOT NULL, FK - Member.id, ON DELETE CASCADE
MemberPlan.plan_id	NOT NULL, FK - WorkoutPlan.id, ON DELETE RESTRICT
Member.email	UNIQUE, NOT NULL